

Operating Systems

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

- (1) **Operating System Concepts** by the Silberschatz, Galvin, Gagne

Please refer to the above books for more details.

Contents

1	Processes	7
1.1	Introduction to Processes	7
1.1.1	Components of a Process	7
1.2	Process State	7
1.2.1	Process State Diagram	8
1.3	Process Control Block	8
1.4	Operations on Processes	8
1.4.1	Process Creation	8
1.4.2	Process Termination	9
1.5	Process Table	9
1.5.1	Trap or Exception	9
1.5.2	Dual-Mode Operation in an Operating System	10
1.5.3	Timers	10
1.5.4	Fork System Call	10
1.5.5	Dual Mode Operation Support	10
1.6	Chapter Summary	10
2	Multithreading	12
2.1	Problems with Traditional Processes	12
2.2	Threads	12
2.3	Types of Threads	12
2.4	Multithreading Models	13
2.5	Benefits of Multithreading	13
3	Process Scheduling	14
3.1	Multiprogramming	14
3.2	Time Sharing	14
3.3	Scheduling Queues	14
3.4	Schedulers	14
3.4.1	Long-term Scheduler or Job Scheduler	15
3.4.2	Short-term Scheduler or CPU Scheduler	15
3.4.3	Medium-term Scheduler	15
3.5	Context Switch	15
3.6	Dispatcher	15
3.7	Preemptive and Nonpreemptive Scheduling	16
3.8	Scheduling Criteria	16

4	Scheduling Algorithms	17
4.1	First-Come, First-Served Scheduling	17
4.2	Shortest-Job-First Scheduling	17
4.2.1	Preemptive SJF Scheduling	17
4.3	Priority Scheduling	17
4.3.1	Aging	18
4.4	Round-Robin Scheduling	18
4.5	SCAN Scheduling	18
4.6	Circular SCAN Scheduling	18
5	Multiprocessor Scheduling	19
5.1	Processor Affinity	19
5.2	Load Balancing	19
6	Deadlocks	20
6.1	Introduction	20
6.2	Necessary Conditions for Deadlock	20
6.3	Resource Allocation Graph	20
6.3.1	Request Edge	21
6.3.2	Assignment Edge	21
6.3.3	Observations	21
6.4	Methods for Handling Deadlocks	21
6.5	Deadlock Prevention	21
6.6	Deadlock Avoidance	22
6.6.1	Safe State	22
6.6.2	Banker's Algorithm	22
6.7	Recovery from Deadlock	23
7	Deadlocks	24
7.1	Introduction	24
7.2	Necessary Conditions for Deadlock	24
7.3	Resource Allocation Graph	24
7.3.1	Request Edge	25
7.3.2	Assignment Edge	25
7.3.3	Observations	25
7.3.4	Shared Memory Model	25
7.4	Message-Passing Systems	26
7.4.1	Synchronous Communication	26
7.4.2	Asynchronous Communication	26
7.4.3	Blocking Send	26
7.4.4	Nonblocking Send	26
7.4.5	Blocking Receive	26
7.4.6	Nonblocking Receive	26
7.4.7	Direct Communication	27
7.4.8	Indirect Communication	27
7.5	Producer-Consumer Problem	27
7.6	Race Condition	28

8	The Critical-Section Problem	29
8.1	Introduction	29
8.1.1	Structure of a Process	29
8.2	Solution to the Critical-Section Problem	29
8.3	Peterson's Solution	30
8.3.1	The Structure of Process P_i in Peterson's Solution	30
9	Synchronization Hardware	31
9.1	Introduction	31
9.2	TestAndSet Instruction	31
9.2.1	The TestAndSet Instruction	31
9.3	Swap Instruction	31
9.4	Bounded-Waiting Mutual Exclusion with TestAndSet	32
10	Semaphores	33
10.1	Introduction	33
10.2	Binary Semaphores	33
10.3	Counting Semaphores	33
10.4	Semaphore Implementation	34
10.4.1	Wait Operation Implementation	34
10.4.2	Signal Operation Implementation	34
10.5	Deadlocks and Starvation	35
11	Code for Producer and Consumer	36
11.1	Producer-Consumer Problem with Race Condition	36
11.1.1	Code for Consumer	36
11.2	Modified Code for Producer	36
11.3	Modified Code for Consumer	37
12	File Systems	38
12.1	File System Structure	38
12.1.1	Levels of File System	38
12.2	I/O Control	38
12.3	Basic File System	38
12.4	File Organization Module	38
12.5	Logical File System	39
12.6	File Systems Used by Various Operating Systems	39
12.7	On-Disk and In-Memory Structures	39
13	Disk Allocation Methods	40
13.1	Contiguous Allocation	40
14	File Organization and Indexes	41
14.1	File Organization	41
14.1.1	Spanned Records	41
14.1.2	Unspanned Records	41
14.1.3	Blocking Factor	41
14.2	Types of Organization of Files	41
14.3	Indexes	42

14.3.1	Dense and Sparse Indexes	42
14.3.2	Types of Indexes	42
14.4	Single-Level Ordered Indexes	42
14.4.1	Primary Indexes	42
14.4.2	Clustering Indexes	42
14.4.3	Secondary Indexes	43
14.5	Multilevel Indexes	43
14.6	B-Trees	43
14.6.1	Introduction to B-Trees	43
14.6.2	B+ Tree Structure	44
14.6.3	Insertion in B+ Trees	45
14.6.4	Deletion in B+ Trees	46
14.6.5	Shortest Seek Time First Scheduling	47
14.6.6	SCAN Scheduling	47
14.6.7	Circular SCAN Scheduling	47
14.7	Disk Formatting	47
14.7.1	Physical Formatting	48
14.7.2	Logical Formatting	48
14.8	Raw Disk	48
14.9	Disk Block	48

Chapter 1

Processes

1.1 Introduction to Processes

A **process** is a program in execution. A program by itself is not a process; a program is a passive entity, whereas a process is an active entity.

1.1.1 Components of a Process

A process includes the following:

- (i) **Text Section:** Contains the program code.
- (ii) **Current Activity:** Represented by the value of the program counter and the contents of the processor's registers.
- (iii) **Stack:** Contains temporary data such as function parameters, return addresses, and local variables.
- (iv) **Data Section:** Contains global variables.
- (v) **Heap:** Used for dynamic memory allocation during process runtime.

1.2 Process State

Each process may be in one of the following states during execution:

- (i) **New:** The process is being created.
- (ii) **Running:** Instructions are being executed.
- (iii) **Waiting:** The process is waiting for some event to occur, such as I/O completion or reception of a signal.
- (iv) **Ready:** The process is waiting to be assigned to a processor.
- (v) **Terminated:** The process has finished execution.

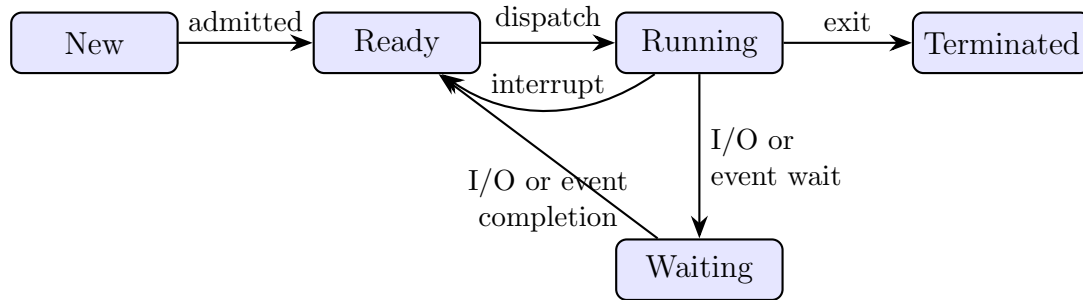


Figure 1.1: Process State Transition Diagram

1.2.1 Process State Diagram

The transitions between process states can be represented as:

Note: Only one process can be running on any processor at any instant.

1.3 Process Control Block

Each process is represented by a process control block (PCB), also called a task control block. A PCB contains the following information:

- (1) **Process State:** Current state of the process.
- (2) **Program Counter:** Address space for the process.
- (3) **CPU Registers:** User identification.
- (4) **List of Open Files:** Files currently accessed by the process.
- (5) **CPU Scheduling Information:** Priority and scheduling parameters.
- (6) **Memory Management Information:** Base and limit registers, page tables.
- (7) **Accounting Information:** Resource usage statistics.
- (8) **I/O Status:** List of I/O devices allocated to the process.
- (9) **Process Identifier (PID):** Unique identifier for the process.
- (10) **Parent Process ID (PPID):** Identifier of the parent process.

1.4 Operations on Processes

1.4.1 Process Creation

A process may create several new processes via a create-process system call during the course of execution.

- The creating process is called a **parent process**, and the new processes are called the **children** of that process.

- Each of these processes may in turn create other processes, forming a tree of processes.
- When a process creates a subprocess, the subprocess may be able to obtain its resources directly from the operating system, or it may be constrained to a subset of the resources of the parent process.
- Restricting a child process to a subset of the parent's resources prevents any process from overloading the system by creating too many subprocesses.

In UNIX systems:

- A new process is created by the `fork()` system call.
- The `exec()` system call is used after a `fork()` system call to replace the process's memory space with a new program.

In Windows systems:

- A new process is created using the `CreateProcess()` function.

1.4.2 Process Termination

A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the `exit()` system call.

- Some systems do not allow a child to exist if its parent process has terminated. In such systems, if a process terminates, then all its children must also be terminated. This phenomenon is referred to as **cascading termination**.
- In UNIX, if the parent terminates, all its children are assigned `init` process as their parent.

1.5 Process Table

The process table is a data structure maintained by the operating system to facilitate context switching and scheduling. Each entry in the process table (an entry is known as a process control block PCB) contains:

- Process key (an entry in the process table).
- Entries are known as process control blocks (PCBs).

1.5.1 Trap or Exception

A **trap** is a software-generated interrupt caused either by an error or by a specific request from a user program.

1.5.2 Dual-Mode Operation in an Operating System

There are two separate modes of operation in an operating system:

- (i) **User Mode**
- (ii) **Kernel Mode** or **Supervisor Mode** or **System Mode** or **Privileged Mode**

Mode Bit

A mode bit is used to distinguish between user mode and kernel mode:

- For user mode: mode-bit = 1
- For kernel mode: mode-bit = 0

At system boot time, the hardware starts in kernel mode. The operating system is then loaded and starts user applications in user mode.

Whenever a trap or interrupt occurs, the hardware switches from user mode to kernel mode.

1.5.3 Timers

A timer is used to ensure that the operating system maintains control over the CPU. A timer can be set to interrupt the computer after a specified period. We can use the timer to prevent a user program from running too long.

1.5.4 Fork System Call

When a process forks, it creates a copy of itself. In the child process, the return value of the `fork()` is zero. In the parent process, the return value is the PID of the newly created child process. There is a separate address space for the child.

Both the parent and child processes pass the same code segments, but execute independently of each other.

For example, for the program with n `fork()` calls, $2^n - 1$ child processes will be created.

1.5.5 Dual Mode Operation Support

Dual mode operation support is provided by the hardware, not the software.

1.6 Chapter Summary

- At system boot time, the hardware starts in kernel mode, then loads the operating system in kernel mode, and starts user applications in user mode.
- Whenever a trap or interrupt occurs, the hardware switches from user mode to kernel mode.
- The dual mode of operation provides us with the means for protecting the operating system from errant users and errant users from one another.

- Some of the machine instructions that may cause harm are designated as privileged instructions. The hardware allows privileged instructions to be executed only in kernel mode.
- If an attempt is made to execute a privileged instruction in user mode, the hardware treats it as illegal and traps it to the operating system.
- System calls provide the means for a user program to ask the operating system to perform tasks reserved for the operating system on the user program's behalf.
- When a system call is executed, it is treated by the hardware as a software interrupt.
- System calls provide an interface between a process and the operating system to ask for operating system services or to perform tasks reserved for the operating system on the user program's behalf.

Chapter 2

Multithreading

2.1 Problems with Traditional Processes

Traditional processes have separate address spaces:

- (i) Process creation is expensive.
- (ii) Process switch is expensive.
- (iii) Sharing memory areas among processes is non-trivial.
- (iv) A process is a single thread of execution.

2.2 Threads

A **thread** is a basic unit of CPU utilization. It comprises:

- (i) Thread ID
- (ii) Program Counter (PC)
- (iii) A register set
- (iv) A stack

It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.

Many operating system kernels are now multithreaded.

2.3 Types of Threads

There are two types:

- (i) **User-Level Threads:** Managed by thread library. Kernel is not aware of threads.
- (ii) **Kernel-Level Threads:** All thread management done in kernel. Kernel is aware of threads.

2.4 Multithreading Models

There must exist a relationship between user threads and kernel threads. There are three common ways of establishing this relationship:

- (i) **Many-to-One Model:** Maps many user-level threads to one kernel thread.
- (ii) **One-to-One Model:** Maps each user thread to a kernel thread. It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call.
- (iii) **Many-to-Many Model:** It multiplexes many user-level threads to a smaller or equal number of kernel threads.

2.5 Benefits of Multithreading

- (i) **Responsiveness:** Multithreading an interactive application may allow a program to continue running even if part of it is blocked, thereby increasing responsiveness to the user.
- (ii) **Resource Sharing:** Threads share the address space and the resources of the process to which they belong.
- (iii) **Economy:** Thread creation is much simpler than process creation. Thread switch is simpler.
- (iv) **Utilization of Multiprocessor Architectures:** Threads can run in parallel on different processors.

Chapter 3

Process Scheduling

3.1 Multiprogramming

The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization. In multiprogrammed systems, the user cannot interact with the system.

3.2 Time Sharing

The objective of time-sharing (also called multitasking systems) is to switch the CPU among processes so frequently that users can interact with each program while it is running.

For a time-sharing system, the response time should be short.

Time-sharing systems internally use multiprogramming.

3.3 Scheduling Queues

A process migrates among the various scheduling queues throughout its lifetime. The schedulers are used to select the processes from these queues in some fashion.

- (i) **Job Queue:** As processes enter the system, they are put into a job queue, which consists of all processes in the system.
- (ii) **Ready Queue:** The processes that are residing in main memory and are ready and waiting to execute are kept on a list called the ready queue.
- (iii) **Device Queue:** The list of processes waiting for a particular I/O device is called a device queue. Each device has its own device queue.

3.4 Schedulers

A process migrates among the various scheduling queues throughout its lifetime. The schedulers are used to select the processes from these queues in some fashion.

There are generally three types of schedulers:

- (1) **Long-term Scheduler or Job Scheduler**

(2) **Short-term Scheduler or CPU Scheduler**

(3) **Medium-term Scheduler**

3.4.1 Long-term Scheduler or Job Scheduler

The long-term scheduler selects processes from the job queue and loads them into memory for execution. The long-term scheduler controls the degree of multiprogramming.

3.4.2 Short-term Scheduler or CPU Scheduler

The short-term scheduler selects from among the processes that are ready to execute and allocates the CPU to one of them. The short-term scheduler must select a new process for the CPU frequently and it must be fast.

3.4.3 Medium-term Scheduler

The key idea behind a medium-term scheduler is that sometimes it can be advantageous to remove processes from memory (and thus reduce the degree of multiprogramming). Later, the process can be reintroduced into memory, and its execution can be continued where it left off. This scheme is called **swapping**. The process is swapped out, and is later swapped in, by the medium-term scheduler.

On some systems, the long-term scheduler may be absent or minimal, such as some time-sharing systems.

The medium-term scheduler is generally used in time-sharing systems.

3.5 Context Switch

Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process. This task is known as a **context switch**.

Context-switch time is pure overhead, because the system does no useful work while switching.

Its speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied, and the existence of special instructions (such as a single instruction to load or store all registers).

3.6 Dispatcher

The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler. This function involves:

- (i) Switching context
- (ii) Switching to user mode
- (iii) Jumping to the proper location in the user program to restart the program

The dispatcher should be as fast as possible.

The time taken by the dispatcher to stop one process and start another running is known as the **dispatch latency**.

3.7 Preemptive and Nonpreemptive Scheduling

Under nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.

Nonpreemptive scheduling is also called **cooperative scheduling**.

Whenever a process switches from the running state to the ready state or waiting state to the ready state, and if scheduling takes place in these cases, then the scheduling is called **preemptive scheduling**.

A scheduling is preemptive if once a process has been given the CPU, it can be taken away.

Preemptive scheduling incurs a cost associated with access to shared data. The shared data can become inconsistent if one process that is updating some data is preempted by another process, and after that another process tries to read this data.

3.8 Scheduling Criteria

It is desirable to maximize CPU utilization and throughput, and to minimize turnaround time, waiting time, and response time.

- (i) **CPU Utilization:** The CPU should be as busy as possible.
- (ii) **Throughput:** The number of processes that are completed per unit time is called throughput.
- (iii) **Turnaround Time:** The interval from the time of submission of a process to the time of completion is the turnaround time.
- (iv) **Waiting Time:** Waiting time is the sum of the periods spent waiting in the ready queue.
- (v) **Response Time:** The time from the submission of a request until the first response is produced is called response time.

Chapter 4

Scheduling Algorithms

4.1 First-Come, First-Served Scheduling

The process that requests the CPU first is allocated to the CPU first. It is implemented using a FIFO queue.

FCFS scheduling algorithm is non-preemptive. The average waiting time in FCFS is often quite long.

4.2 Shortest-Job-First Scheduling

This algorithm associates with each process the length of the process's next CPU burst. When the CPU is available, it is assigned to the process that has the smallest next CPU burst.

If the next CPU bursts of two processes are the same, the FCFS scheduling is used to break the tie.

The SJF algorithm is optimal, in that it gives the minimum average waiting time for a given set of processes.

The SJF algorithm cannot be implemented at the level of short-term CPU scheduling. There is no way to know the length of the next CPU burst.

The SJF algorithm can be either preemptive or nonpreemptive.

4.2.1 Preemptive SJF Scheduling

Preemptive SJF scheduling is sometimes called **Shortest-Remaining-Time-First Scheduling**.

4.3 Priority Scheduling

A priority is associated with each process, and the CPU is allocated to the process with the highest priority.

Equal-priority processes are scheduled in FCFS order.

Priority scheduling can be either preemptive or nonpreemptive.

4.3.1 Aging

A preemptive priority scheduling algorithm will preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.

A major problem with priority scheduling algorithms is indefinite blocking or starvation.

A solution to the problem of indefinite blocking of low-priority processes is **aging**.

Aging is a technique of gradually increasing the priority of processes that wait in the system for a long time.

4.4 Round-Robin Scheduling

The round-robin scheduling algorithm is designed especially for time-sharing systems.

It is similar to FCFS scheduling, but preemption is added to switch between processes. The ready queue is treated as a circular queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

4.5 SCAN Scheduling

The disk arm starts at one end of the disk and moves towards the other end, servicing requests as it reaches each cylinder. At the other end, the direction of head movement is reversed. It is sometimes called the elevator algorithm.

4.6 Circular SCAN Scheduling

It is similar to SCAN scheduling. However, when the head reaches the other end, it immediately returns to the beginning of the disk without servicing any requests on the return trip.

Chapter 5

Multiprocessor Scheduling

There are two approaches to multiprocessor scheduling:

- (1) **Asymmetric Multiprocessing**: All scheduling decisions, I/O processing, and other system activities are handled by a single processor, the master server.
- (2) **Symmetric Multiprocessing (SMP)**: Here, each processor is self-scheduling.

5.1 Processor Affinity

When a process migrates from one processor to another processor, the contents of the cache memory must be invalidated for the processor being migrated from, and the cache must be repopulated for the processor being migrated to.

Because of the high cost of invalidating and re-populating caches, most SMP systems try to avoid migration of processes and instead attempt to keep a process running on the same processor. This is known as **processor affinity**.

5.2 Load Balancing

Load balancing attempts to keep the workload evenly distributed across all processors in an SMP system. The load balancing is typically only necessary on systems where each processor has its own private queue of eligible processes to execute.

There are two approaches to load balancing:

- (1) **Push Migration**: A specific task periodically checks the load on each processor and, if it finds an imbalance, evenly distributes the load by moving processes from overloaded to idle or less-busy processors.
- (2) **Pull Migration**: It occurs when an idle processor pulls a waiting task from a busy processor.

Push and pull migration need not be mutually exclusive and are often implemented in parallel.

Chapter 6

Deadlocks

6.1 Introduction

A set of processes is in a deadlock state when every process in the set is waiting for an event that can be caused only by another process in the set.

6.2 Necessary Conditions for Deadlock

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

- (1) **Mutual Exclusion:** At least one resource must be held in a non-sharable mode; that is, only one process at a time can use the resource.
- (2) **Hold and Wait:** A process must be holding at least one resource and waiting for acquiring additional resources that are currently being held by other processes.
- (3) **No Preemption:** Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- (4) **Circular Wait:** A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , and P_n is waiting for a resource held by P_0 .

These four conditions are not completely independent.

6.3 Resource Allocation Graph

A **resource allocation graph** is used for describing deadlocks precisely.

The graph consists of a set of vertices V and a set of edges E . The set of vertices V is partitioned into two different types of nodes:

- $P = \{P_1, P_2, \dots, P_n\}$: The set consisting of all the active processes in the system.
- $R = \{R_1, R_2, \dots, R_m\}$: The set consisting of all resource types in the system.

6.3.1 Request Edge

A directed edge from process P_i to resource type R_j , denoted by $P_i \rightarrow R_j$, is called a request edge. It signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.

6.3.2 Assignment Edge

A directed edge from resource type R_j to process P_i , denoted by $R_j \rightarrow P_i$, is called an assignment edge. It signifies that an instance of resource type R_j has been allocated to process P_i .

6.3.3 Observations

- If the graph contains no cycles, then no process in the system is deadlocked.
- If the graph does contain a cycle, then a deadlock may exist.
- If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred.
- If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred.

6.4 Methods for Handling Deadlocks

There are three ways to deal with the deadlock problem:

- (i) We can use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlock state.
- (ii) We can allow the system to enter a deadlock state, detect it, and recover.
- (iii) We can ignore the problem altogether and pretend that deadlocks never occur in the system.

6.5 Deadlock Prevention

By ensuring that at least one of these conditions cannot hold, we can prevent the occurrence of a deadlock:

- (i) **Mutual Exclusion:** At least one resource must be held in a non-sharable mode. That is, only one process at a time can use the resource.
- (ii) **Hold and Wait:** A process must be holding at least one resource and waiting for acquiring additional resources that are currently being held by other processes.
- (iii) **No Preemption:** Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.

- (iv) **Circular Wait:** A set $\{P_0, P_1, P_2, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , and P_n is waiting for a resource held by P_0 .

The drawback of this method is low device utilization and reduced system throughput.

6.6 Deadlock Avoidance

A deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular wait condition can never exist. The resource-allocation state is defined by the number of available and allocated resources and the maximum demands of the processes.

There are two algorithms for deadlock avoidance:

- (i) Resource-Allocation-Graph Algorithm
- (ii) Banker's Algorithm

6.6.1 Safe State

A state is safe if the system can allocate resources to each process in some order and still avoid a deadlock. More formally, a system is in a safe state only if there exists a safe sequence.

A sequence of processes $\langle P_1, P_2, \dots, P_n \rangle$ is a safe sequence for the current allocation state if, for each P_i , the resource requests that P_i can still make can be satisfied by the currently available resources plus the resources held by all P_j , with $j < i$.

A safe state is not a deadlocked state. Conversely, a deadlocked state is an unsafe state.

Not all unsafe states are deadlocks; however, an unsafe state may lead to a deadlock.

6.6.2 Banker's Algorithm

This algorithm is used when a resource type may have multiple instances.

When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need. This number may not exceed the total number of resources in the system.

When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise, the process must wait until some other process releases enough resources.

Let n be the number of processes in the system and m be the number of resource types. We need the following data structures:

- (i) **Available:** A vector of length m indicates the number of available resources of each type. If $\text{Available}[j] = k$, there are k instances of resource type R_j available.
- (ii) **Max:** An $n \times m$ matrix defines the maximum demand of each process. If $\text{Max}[i][j] = k$, then process P_i may request at most k instances of resource type R_j .

- (iii) **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process. If $\text{Allocation}[i][j] = k$, then process P_i is currently allocated k instances of resource type R_j .
- (iv) **Need:** An $n \times m$ matrix indicates the remaining resource need of each process. If $\text{Need}[i][j] = k$, then process P_i may need k more instances of resource type R_j to complete its task.

Note: $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$

6.7 Recovery from Deadlock

When a deadlock detection algorithm determines that a deadlock exists, several alternatives are available:

- (1) **Process Termination:** Two methods are available:
 - Abort all deadlocked processes.
 - Abort one process at a time until the deadlock cycle is eliminated.
- (2) **Resource Preemption:** To eliminate deadlocks using resource preemption, we successively preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken.
- (3) **Rollback:** If we preempt a resource from a process, we must rollback the process to some safe state and restart it from that state.
- (4) **Starvation:** How can we guarantee that resources will not always be preempted from the same process?

Chapter 7

Deadlocks

7.1 Introduction

A set of processes is in a deadlock state when every process in the set is waiting for an event that can be caused only by another process in the set.

7.2 Necessary Conditions for Deadlock

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

- (1) **Mutual Exclusion:** At least one resource must be held in a non-sharable mode; that is, only one process at a time can use the resource.
- (2) **Hold and Wait:** A process must be holding at least one resource and waiting for acquiring additional resources that are currently being held by other processes.
- (3) **No Preemption:** Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- (4) **Circular Wait:** A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , and P_n is waiting for a resource held by P_0 .

These four conditions are not completely independent.

7.3 Resource Allocation Graph

A **resource allocation graph** is used for describing deadlocks precisely.

The graph consists of a set of vertices V and a set of edges E . The set of vertices V is partitioned into two different types of nodes:

- $P = \{P_1, P_2, \dots, P_n\}$: The set consisting of all the active processes in the system.
- $R = \{R_1, R_2, \dots, R_m\}$: The set consisting of all resource types in the system.

7.3.1 Request Edge

A directed edge from process P_i to resource type R_j , denoted by $P_i \rightarrow R_j$, is called a request edge. It signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.

7.3.2 Assignment Edge

A directed edge from resource type R_j to process P_i , denoted by $R_j \rightarrow P_i$, is called an assignment edge. It signifies that an instance of resource type R_j has been allocated to process P_i .

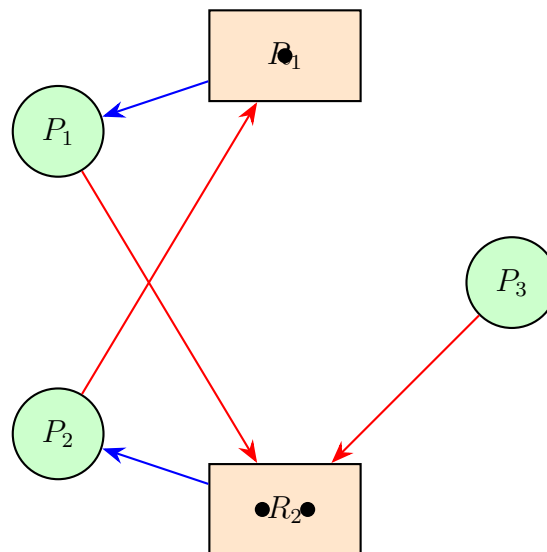


Figure 7.1: Resource Allocation Graph Example (With Cycle - Possible Deadlock)

7.3.3 Observations

- If the graph contains no cycles, then no process in the system is deadlocked.
- If the graph does contain a cycle, then a deadlock may exist.
- If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred.
- If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred.

7.3.4 Shared Memory Model

In this model, a region of memory that is shared by cooperating processes is established. Typically, a shared-memory region resides in the address space of the process creating the shared-memory segment. Other processes that wish to communicate using this shared-memory segment must attach it to their address space.

The processes are also responsible for ensuring that they are not writing to the same location simultaneously.

7.4 Message-Passing Systems

Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space. It is particularly useful in a distributed environment where the communicating processes may reside on different computers connected by a network.

A message-passing facility provides at least two operations:

- (i) Send (message)
- (ii) Receive (message)

The communication between different processes in a message-passing system can be of the following types:

- (a) **Synchronous or Asynchronous Communication**
- (b) **Direct or Indirect Communication**

7.4.1 Synchronous Communication

In synchronous communication, processes synchronize at every message. Both send and receive are blocking operations.

7.4.2 Asynchronous Communication

The send operation is almost always non-blocking. The receive operation, however, can be blocking or non-blocking.

7.4.3 Blocking Send

The sending process is blocked until the message is received by the receiving process.

7.4.4 Nonblocking Send

The sending process sends the message and resumes operation.

7.4.5 Blocking Receive

The receiver blocks until a message is available.

7.4.6 Nonblocking Receive

The receiver retrieves either a valid message or a null.

7.4.7 Direct Communication

Processes must explicitly name the receiver or sender of a message (symmetric addressing):

- `send( $P$ , message)`. Send message to process P .
- `receive( $Q$ , message)`. Receive message from Q .

In a client-server system, the server does not have to know the name of a specific client in order to receive a message. In this case, a variant of the receive operation can be used (asymmetric addressing):

- `receive( $ID$ , message)`. Receive a pending message from any process. When a message arrives, ID is set to the name of the sender.

7.4.8 Indirect Communication

With indirect communication, the messages are sent to and received from mailboxes, or ports.

A mailbox can be viewed abstractly as an object into which messages can be placed by processes and from which messages can be removed.

The `send()` and `receive()` primitives are defined as follows:

- `send( $A$ , message)`. Send a message to mailbox A .
- `receive( $A$ , message)`. Receive a message from mailbox A .

Generally, a mailbox is associated with many senders and receivers.

In some systems, only one receiver is associated with a particular mailbox; such a mailbox is often called a **port**.

7.5 Producer-Consumer Problem

A producer process produces information that is consumed by a consumer process.

One solution to the producer-consumer problem uses shared memory. To allow producer and consumer processes to run concurrently, we must have available a buffer of items that can be filled by the producer and emptied by the consumer.

The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced.

Two types of buffers can be used:

- (i) **Unbounded Buffer**: It places no practical limit on the size of the buffer. The producer can always produce new items, but the consumer may have to wait for new items.
- (ii) **Bounded Buffer**: It assumes a fixed buffer size. In this case, the consumer must wait if the buffer is empty, and the producer must wait if the buffer is full.

7.6 Race Condition

A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition**.

To guard against the race condition, we need to ensure that only one process at a time can be manipulating the variable counter. To make such a guarantee, we require that the processes be synchronized in some way.

Chapter 8

The Critical-Section Problem

8.1 Introduction

Consider a system consisting of n processes $\{P_0, P_1, \dots, P_{n-1}\}$. Each process has a segment of code, called a **critical section**, in which the process may be changing common variables, updating a table, writing a file, and so on.

The important feature of the system is that, when one process is executing in its critical section, no other process is to be allowed to execute in its critical section.

The **critical-section problem** is to design a protocol that the processes can use to cooperate. Each process must request permission to enter its critical section.

8.1.1 Structure of a Process

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```

8.2 Solution to the Critical-Section Problem

A solution to the critical-section problem must satisfy the following three requirements:

- (1) **Mutual Exclusion:** If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
- (2) **Progress:** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in the decision on which will enter its critical section next, and this selection cannot be postponed indefinitely.
- (3) **Bounded Waiting:** There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Two general approaches are used to handle critical sections in operating systems:

- (i) **Preemptive Kernels:** Allows a process to be preempted while it is running in kernel mode.
- (ii) **Nonpreemptive Kernels:** Does not allow a process running in kernel mode to be preempted; it will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU.

A nonpreemptive kernel is essentially free from race conditions on kernel data structures, as only one process is active in the kernel at a time.

8.3 Peterson's Solution

Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections. The processes are numbered P_i and P_j , where $j = 1 - i$.

Peterson's solution requires two data items to be shared between the two processes:

```
int turn;
boolean flag[2];
```

The variable `turn` indicates whose turn it is to enter the critical section.

The `flag` array is used to indicate that a process is ready to enter its critical section.

8.3.1 The Structure of Process P_i in Peterson's Solution

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);
    critical section
    flag[i] = false;
    remainder section
} while (true);
```

To enter the critical section, process P_i first sets `flag[i]` to be true and then sets `turn` to the value j , thereby asserting that if the other process wishes to enter the critical section, it can do so.

If both processes try to enter at the same time, `turn` will be set to both i and j at roughly the same time. Only one of these assignments will last; the other will occur but will be overwritten immediately. The eventual value of `turn` decides which of the two processes is allowed to enter its critical section first.

Peterson's solution is a software-based solution.

Chapter 9

Synchronization Hardware

9.1 Introduction

In general, we can state that any solution to the critical-section problem requires a simple tool, a **lock**. Race conditions are prevented by requiring that critical regions be protected by locks.

9.2 TestAndSet Instruction

The important characteristic of this instruction is that it is executed **atomically**. Thus, if two `TestAndSet()` instructions are executed simultaneously (each on a different CPU), they will be executed sequentially in some arbitrary order.

9.2.1 The TestAndSet Instruction

```
boolean TestAndSet(boolean *lock) {  
    boolean rv = *lock;  
    *lock = true;  
    return rv;  
}
```

If the machine supports the `TestAndSet()` instruction, then mutual exclusion can be provided.

9.3 Swap Instruction

The `Swap()` instruction operates on the contents of two words. Like the `TestAndSet()` instruction, it is executed atomically.

```
void Swap(boolean *lock, boolean *key) {  
    boolean temp = *lock;  
    *lock = *key;  
    *key = temp;  
}
```

9.4 Bounded-Waiting Mutual Exclusion with TestAnd-Set

```
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = TestAndSet(&lock);
    waiting[i] = false;

    critical section

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;

    if (j == i)
        lock = false;
    else
        waiting[j] = false;

    remainder section
} while (true);
```

For the algorithm using the `TestAndSet()` instruction that satisfies all the critical-section requirements, the common data structures are:

```
boolean waiting[n];
boolean lock;
```

These data structures are initialized to `false`.

Chapter 10

Semaphores

10.1 Introduction

The various hardware-based solutions to the critical-section problem are complicated as well as generally inaccessible to application programmers. To overcome this difficulty, we can use a synchronization tool called a **semaphore**.

A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: `wait()` and `signal()`.

The `wait()` operation was originally termed P and `signal()` was originally called V .

The definition of `wait()` is as follows:

```
wait(S) {
    while (S <= 0)
        ; /* No operation */
    S--;
}
```

The definition of `signal()` is as follows:

```
signal(S) {
    S++;
}
```

10.2 Binary Semaphores

The value of a binary semaphore can range only between 0 and 1. On some systems, binary semaphores are known as **mutex locks**, as they are locks that provide mutual exclusion.

We can use binary semaphores to deal with the critical-section problem for multiple processes. The n processes share a semaphore, **mutex**, initialized to 1.

10.3 Counting Semaphores

The value of a counting semaphore can range over an unrestricted domain. Counting semaphores can be used to control access to a given resource consisting of a finite number

of instances. The semaphore is initialized to the number of resources available. Each process that wishes to use a resource performs a `wait()` operation. When a process releases a resource, it performs a `signal()` operation.

When the count for the semaphore goes to 0, all resources are being used. Processes that wish to use a resource will block until the count becomes greater than 0.

10.4 Semaphore Implementation

The main disadvantage of the semaphore definition given here is that it requires **busy waiting**. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code. This type of semaphore is also called a **spinlock** because the process “spins” while waiting for the lock.

To overcome the need for busy waiting, we can modify the definition of the `wait()` and `signal()` semaphore operations.

10.4.1 Wait Operation Implementation

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}
```

10.4.2 Signal Operation Implementation

When some other process executes a `signal()` operation, which changes the process from the waiting state to the ready state, the process is placed in the ready queue.

```
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

Note: Although under the classical definition of semaphores with busy waiting, the semaphore value is never negative, this implementation may have negative semaphore values. If the semaphore value is negative, its magnitude is the number of processes waiting on that semaphore.

The critical aspect of semaphores is that they be executed atomically. We must guarantee that no two processes can execute `wait()` and `signal()` operations on the same semaphore at the same time.

This is a critical-section problem; and in a single-processor environment, we can solve it by simply inhibiting interrupts during the time the `wait()` and `signal()` operations are executing.

In a multiprocessor environment, disabling interrupts on every processor can be a difficult task. Therefore, SMP systems must provide alternative locking techniques to ensure that `wait()` and `signal()` are performed atomically.

10.5 Deadlocks and Starvation

The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. The event in question is the execution of a `signal()` operation.

When such a state is reached, these processes are said to be **deadlocked**.

Another problem related to deadlocks is **indefinite blocking or starvation**. A situation where processes wait indefinitely within the semaphore. Indefinite blocking may occur if we add and remove processes from the list associated with a semaphore in LIFO order.

Chapter 11

Code for Producer and Consumer

11.1 Producer-Consumer Problem with Race Condition

11.1.1 Code for Consumer

```
item nextConsumed;
while (true) {
    while (in == out)
        ; /* do nothing */
    nextConsumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    /* consume the item in nextConsumed */
}
```

The producer process has a local variable `nextProduced` in which the new item to be produced is stored. The consumer process has a local variable `nextConsumed` in which the item to be consumed is stored.

This scheme allows at most `BUFFER_SIZE - 1` items in the buffer at the same time. One possible remedy is to add an integer variable `counter`, initialized to 0. Counter is incremented every time we add a new item to the buffer and is decremented every time we remove one item from the buffer.

11.2 Modified Code for Producer

```
while (true) {
    /* produce an item in nextProduced */
    while (counter == BUFFER_SIZE)
        ; /* do nothing */
    buffer[in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

11.3 Modified Code for Consumer

```
while (true) {  
    while (counter == 0)  
        ; /* do waiting */  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* Consume the item in nextConsumed */  
}
```

Although both the producer and consumer routines are correct separately, they may not function correctly when executed concurrently.

Suppose that the value of the variable `counter` is currently 5 and that the producer and consumer processes execute the statements `counter++` and `counter--` concurrently. Following the execution of these two statements, the value of the variable `counter` may be 4, 5, or 6!

The only correct result, though, is `counter == 5`, which is generated correctly if the producer and consumer execute separately.

Chapter 12

File Systems

12.1 File System Structure

A file system is used to allow the data to be stored, located, and retrieved easily. The file system itself is generally composed of many different levels.

12.1.1 Levels of File System

- Application Programs
- Logical File System
- File-Organization Module
- Basic File System
- I/O Control
- Devices

12.2 I/O Control

I/O control consists of device drivers and interrupt handlers to transfer information between the main memory and the disk system.

12.3 Basic File System

It needs only to issue generic commands to the appropriate device driver to read and write physical blocks on the disk.

12.4 File Organization Module

It knows about files and their logical blocks as well as physical blocks. It translates logical block addresses to physical block addresses.

12.5 Logical File System

It manages the metadata information. It manages the directory structure via file-control blocks.

A **file control block (FCB)** contains information about the file, including ownership, permissions, and location of the file contents.

12.6 File Systems Used by Various Operating Systems

- (i) **UNIX** → UNIX File System (UFS), which is based on Berkeley Fast File System (FFS).
- (ii) **Windows** → FAT, FAT32, NTFS
- (iii) **Linux** → Extended File System (ext2 & ext3)

12.7 On-Disk and In-Memory Structures

Several on-disk and in-memory structures are used to implement a file system. These structures vary depending on the operating system and the file system, but some general principles apply.

On disk, the file system may contain the following information:

- (i) **Boot Control Block (per volume)**: Contains information needed by the system to boot an OS from that volume. In UFS, it is called the **boot block**. In NTFS, it is called the partition boot sector.
- (ii) **Volume Control Block (per volume)**: Contains volume (or partition) details such as the number of blocks in the partition and the size of blocks, free-block count and free-block pointers. In UFS, it is called a **superblock**. In NTFS, it is stored in the master file table.
- (iii) **Directory Structure (per file system)**: Used to organize the files. In UFS, this includes file names and associated inode numbers. In NTFS, it is stored in the master file table.
- (iv) **File Control Block - FCB (per file)**: Contains details about the file, including file permissions, ownership, size, etc. In UFS, this is called the **inode block**. In NTFS, this information is actually stored within the master file table.

Chapter 13

Disk Allocation Methods

The major methods of allocating disk space are in wide use:

- (1) Contiguous Allocation
- (2) Linked Allocation
- (3) Indexed Allocation

13.1 Contiguous Allocation

Chapter 14

File Organization and Indexes

14.1 File Organization

14.1.1 Spanned Records

If records can span more than one block in a disk, this organization is called spanned.

Whenever a record is larger than a block, we must use a spanned organization.

14.1.2 Unspanned Records

If records are not allowed to cross block boundaries, the organization is called unspanned. This is used with fixed-length records having $B > R$.

14.1.3 Blocking Factor

Suppose that the block size is B bytes. For a file of fixed-length records of size R bytes, with $B \geq R$, we can fit $\text{bfr} = \lfloor B/R \rfloor$ records per block.

The value bfr is called the blocking factor.

The maximum number of records that can fit in a disk block is called the blocking factor (bfr).

14.2 Types of Organization of Files

There are three types of organization of files:

- (1) **Files of Unordered Records (Heap Files):** The records are placed in the file in the order in which they are inserted, so new records are inserted at the end of the file.
- (2) **Files of Ordered Records (Sorted Files):** We can physically order the records of a file on disk based on the values of one of their fields, called the **ordering field**. If the ordering field is also a key field of the file, then the field is called the **ordering key field**.
- (3) **Hash Files:** The idea behind hashing is to provide a function h , called a **hash function**, that is applied to the hash field value of a record and yields the address of the disk block in which the record is stored.

For most records, we need only a single-block access to retrieve that record.
The search condition must be an equality condition on a single field.
All the above organizations of files are called primary organization.

14.3 Indexes

The index structures typically provide secondary access paths to access the records without affecting the physical placement of records on disk.

14.3.1 Dense and Sparse Indexes

A **dense index** has an index entry for every search key value in the search key.

A **sparse index** has index entries for only some of the search values.

14.3.2 Types of Indexes

(1) Single-Level Ordered Indexes

- (a) Primary Indexes
- (b) Clustering Indexes
- (c) Secondary Indexes

(2) Multi-Level Indexes

14.4 Single-Level Ordered Indexes

14.4.1 Primary Indexes

A **primary index** is a non-dense (sparse) index.

A major problem with a primary index—as with any ordered file—is insertion and deletion of records.

14.4.2 Clustering Indexes

If records of a file are physically ordered on a non-key field, that field is called the **clustering field**.

A **clustering index** is also an ordered file with two fields. The first field is of the same data type as some non-ordering field of the data file, and the second field is a block pointer.

There is one entry in the clustering index for each distinct value of the clustering field, containing the value and the pointer to the first block in the data file that has a record with that value.

A clustering index is also a non-dense (sparse) index, because it has an entry for every distinct value of the indexing field, rather than for every record in the file.

14.4.3 Secondary Indexes

It is also an ordered file with two fields. The first field is of the same data type as some non-ordering field of the data file that is an indexing field. The second field is either a block pointer or a record pointer.

There can be many secondary indexes (and hence, secondary key fields) for the same file.

If the secondary index is on a key field, then there is one index entry for each record in the data file. Hence, such an index is dense.

If the secondary index is on a non-key field, then numerous records in the data file can have the same value for the indexing field. There are various options for implementing such an index.

A secondary index provides a logical ordering on the records by the indexing field.

14.5 Multilevel Indexes

A **multilevel index** considers the index file (first level) as an ordered file with a distinct value for each $K(i)$. Hence, we create a primary index for the first level.

The index to the first level is called the second level of the multilevel index, and so on.

We require a second level only if the first level needs more than one block of disk storage and so on.

Each level reduces the number of entries at the previous level by a factor of f_o - the index fan-out.

If the first level has r_1 entries and the blocking factor—which is also called the **fan-out**—of the index is f_o , then the first level needs $\lceil r_1/f_o \rceil$ blocks, which is therefore the number of entries needed at the second level of the index.

14.6 B-Trees

14.6.1 Introduction to B-Trees

The **B-tree** is a search tree with additional constraints that ensure that the tree is always balanced.

A B-tree of order p , when used as an access structure on a key field to search for records in a data file, can be defined as follows:

- (1) Each internal node in the B-tree is of the form:

$$\langle P_1, \langle K_1, P_{r_1} \rangle, P_2, \langle K_2, P_{r_2} \rangle, \dots, \langle K_{q-1}, P_{r_{q-1}} \rangle, P_q \rangle$$

where $q \leq p$. Each P_i is a tree pointer—a pointer to another node in the B-tree. Each P_{r_i} is a data pointer—a pointer to the record whose search key field value is equal to K_i (or to the data file block containing that record).

- (2) Within each node, $K_1 < K_2 < \dots < K_{q-1}$.

(3) For all search key field values X in the subtree pointed at by P_i , we have:

$$\begin{aligned} K_{i-1} < X < K_i & \text{ for } 1 < i < q, \\ X \leq K_i & \text{ for } i = 1, \text{ and} \\ K_{i-1} < X & \text{ for } i = q. \end{aligned}$$

(4) Each node has at most p tree pointers.

(5) Each node, except the root and leaf nodes, has at least $\lceil p/2 \rceil$ tree pointers. The root node has at least two tree pointers unless it is the only node in the tree.

(6) All leaf nodes are at the same level. Leaf nodes have the same structure as internal nodes except that all of their tree pointers P_i are null.

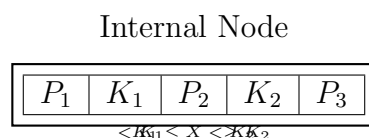


Figure 14.1: B-Tree Node Structure (order p)

14.6.2 B+ Tree Structure

In a B+ tree:

- Internal nodes contain only keys and tree pointers
- Leaf nodes contain keys and data pointers
- All data pointers are at the leaf level
- Leaf nodes are linked together for sequential access

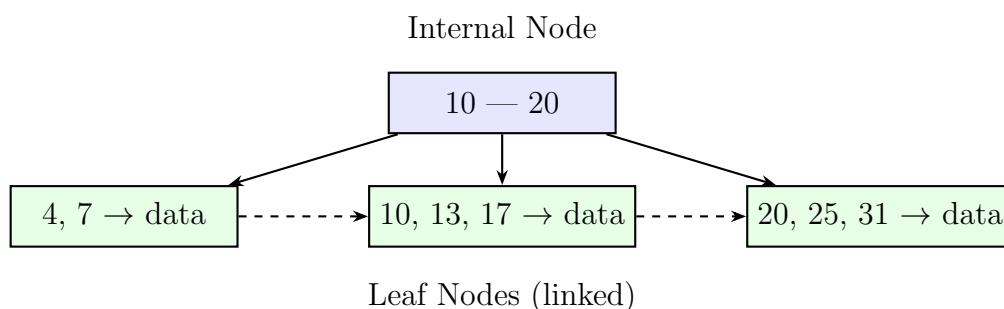


Figure 14.2: B+ Tree Example Structure

14.6.3 Insertion in B+ Trees

Overflow

When the number of search key values exceeds $p - 1$.

Splitting Leaf Node into Two Nodes

- Left node contains $\lceil p/2 \rceil$ values.
- 2nd node contains remaining values.
- Copy the maximum search key value of the 1st node to the parent node.

Example: B+ Tree Insertion with Splitting

Consider inserting values 7, 9, 13, 15 into a B+ tree of order $p = 4$ (can hold 3 keys per node).

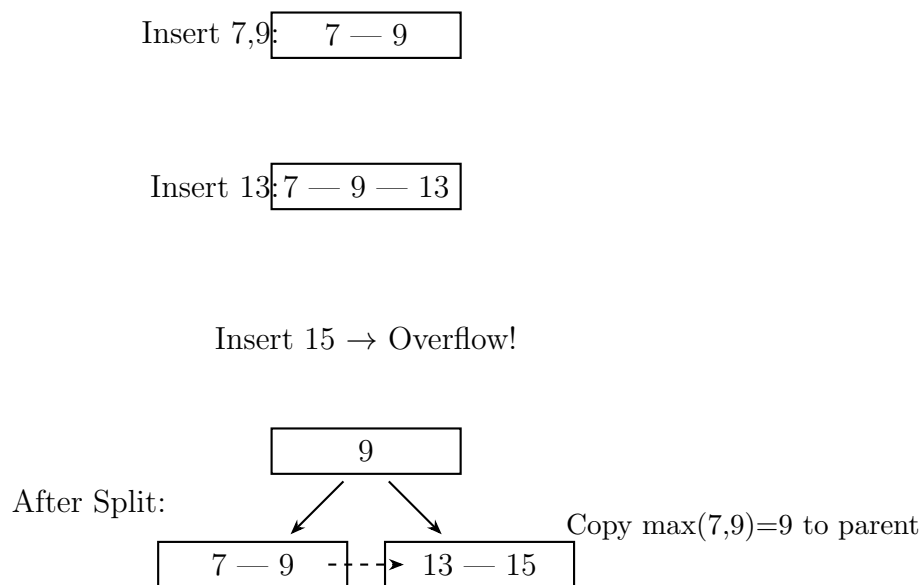


Figure 14.3: B+ Tree Insertion Example with Leaf Node Splitting

Splitting Non-Leaf Node into Two Nodes

- 1st node contains $\lceil p/2 \rceil$ values.
- Move the smallest of the remaining values to parent, with pointer to the parent.
- 2nd node contains the remaining values.

14.6.4 Deletion in B+ Trees

Underflow

When number of search key values $< \lceil p/2 \rceil - 1$.

Leaf Node Deletion

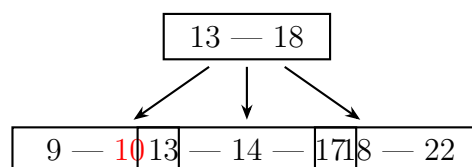
- (a) **Redistribute to sibling:** Right node should not be less than left node.
- (b) Move the smallest value of the right node to left node.
- (c) Replace the between value in parent by the smallest value of the right node.

Merge Operation

- (a) Both left and right nodes have $\lceil (p-1)/2 \rceil$ values.
- (b) Move all values, pointers to left node.
- (c) Delete the right node and pointers in parent.

Example: B+ Tree Deletion with Redistribution

Before: Delete 10



After: Underflow! Redistribute from right sibling

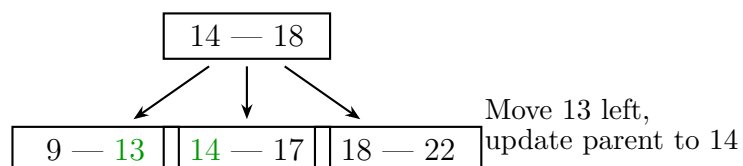


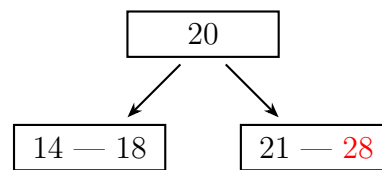
Figure 14.4: B+ Tree Deletion Example with Redistribution

Example: B+ Tree Deletion with Merge

Non-Leaf Node Deletion

- (a) Redistribute to sibling: Through parent.
- (b) Right node should not be less than left node.

Before: Delete 28



After: Underflow! Merge with sibling

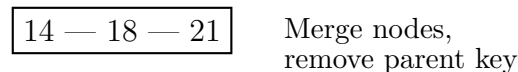


Figure 14.5: B+ Tree Deletion Example with Merge

Merge for Non-Leaf Nodes

- (a) Both left and right node have $\lceil p/2 \rceil$ values.
- (b) Move all values, pointers to left node.
- (c) Delete the right node, and pointers in parent.

14.6.5 Shortest Seek Time First Scheduling

The SSTF algorithm selects the request with the minimum seek time from the head position. It may cause starvation of some requests.

This algorithm is not optimal.

14.6.6 SCAN Scheduling

The disk arm starts at one end of the disk and moves towards the other end, servicing requests as it reaches each cylinder. At the other end, the direction of head movement is reversed. It is sometimes called the elevator algorithm.

14.6.7 Circular SCAN Scheduling

It is similar to SCAN scheduling. However, when the head reaches the other end, it immediately returns to the beginning of the disk without servicing any requests on the return trip.

14.7 Disk Formatting

A new magnetic disk is a blank slate: it is just a platter of a magnetic recording material.

14.7.1 Physical Formatting

Before a disk can store data, it must be divided into sectors that the disk controller can read and write. This process is called **low-level formatting** or **physical formatting**. It fills the disk with a special data structure for each sector.

14.7.2 Logical Formatting

Here, the operating system stores the initial file-system data structures onto the disk.

These data structures may include maps of free and allocated space and an initial empty directory.

14.8 Raw Disk

Some operating systems give special programs the ability to use a disk partition as a large sequential array of logical blocks, without any file-system data structures. This array is sometimes called **raw disk**, and I/O to this array is termed **raw I/O**.

14.9 Disk Block

A disk block is the unit of data transfer between disk and memory.

Blocks must be multiple of sector size. Block size cannot exceed the size of track.